

GENERATIVE AI · SELF-ATTENTION

Outline

Self-Attention의 계산 원리: 같은 문장의 벡터를 비교하고, 기여 비율로 섞은 결과를 검산한다.

입력과 전달물

- 입력은 여섯 토큰의 3차원 예시 벡터다. 토큰은 여기서 단어 하나에 대응한다.
- 각 토큰에 문맥 정보를 섞은 벡터를 만들고 `attention_report.py` 로 동료에게 계산·검산 결과를 전달한다.

계산과 성공 기준

- 비교 점수 → 기여 비율 → 문맥 벡터를 계산한 뒤 전체 토큰으로 확장한다.
- 입력과 출력은 [6,3], 기여 비율은 [6,6]이다. 각 행 합 1과 단일·전체 계산의 일치를 검산한다.
- 학습 파라미터 없는 self-attention을 다룬다. 투영 행렬·미래 입력 차단·여러 attention의 병렬 구성은 포함하지 않는다.

단일 Hidden State의 정보 병목

입력을 압축하는 구성

- 순환 신경망(RNN)은 이전 내부 상태와 현재 토큰을 읽어 새 상태를 만드는 신경망이다.
- 그림의 encoder는 독일어 입력을 읽는 부분, decoder는 그 상태로 영어 출력을 만드는 부분이다.
- 회색 상자는 중간 상태, 마지막 연결선은 입력 전체를 요약한 한 상태의 전달을 뜻한다.

접근 경로의 제약

- 이 그림의 decoder에는 앞선 입력별 상태를 직접 다시 읽는 경로가 없다. 긴 문장의 단서가 약해질 수 있다.
- 이는 모든 RNN의 필연적 실패가 아니라, 마지막 상태만 전달하는 구성의 제약이다.

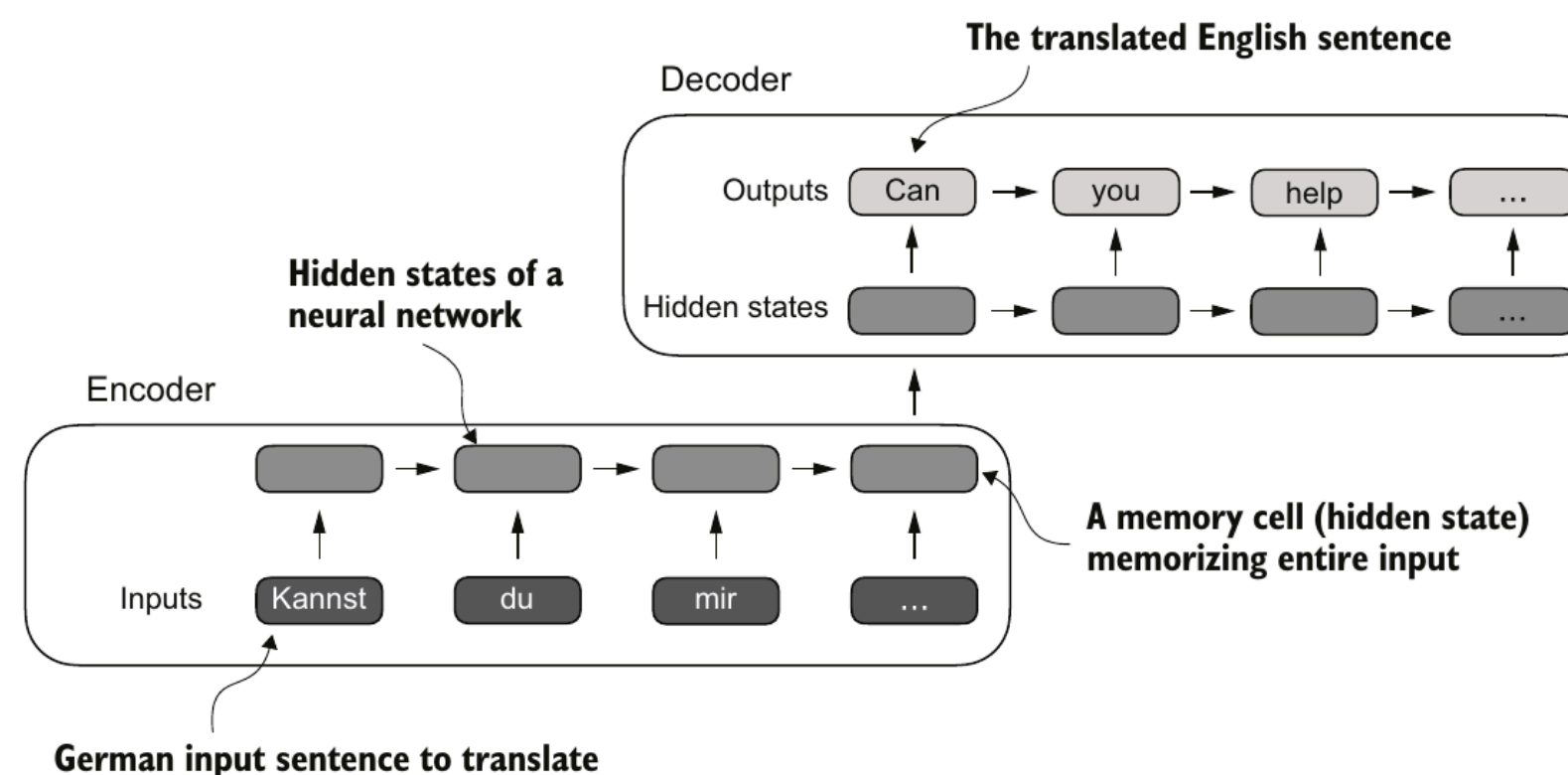


Figure 3.4 Before the advent of transformer models, encoder–decoder RNNs were a popular choice for machine translation. The encoder takes a sequence of tokens from the source language as input, where a hidden state (an intermediate neural network layer) of the encoder encodes a compressed representation of the entire input sequence. Then, the decoder uses its current hidden state to begin the translation, token by token.

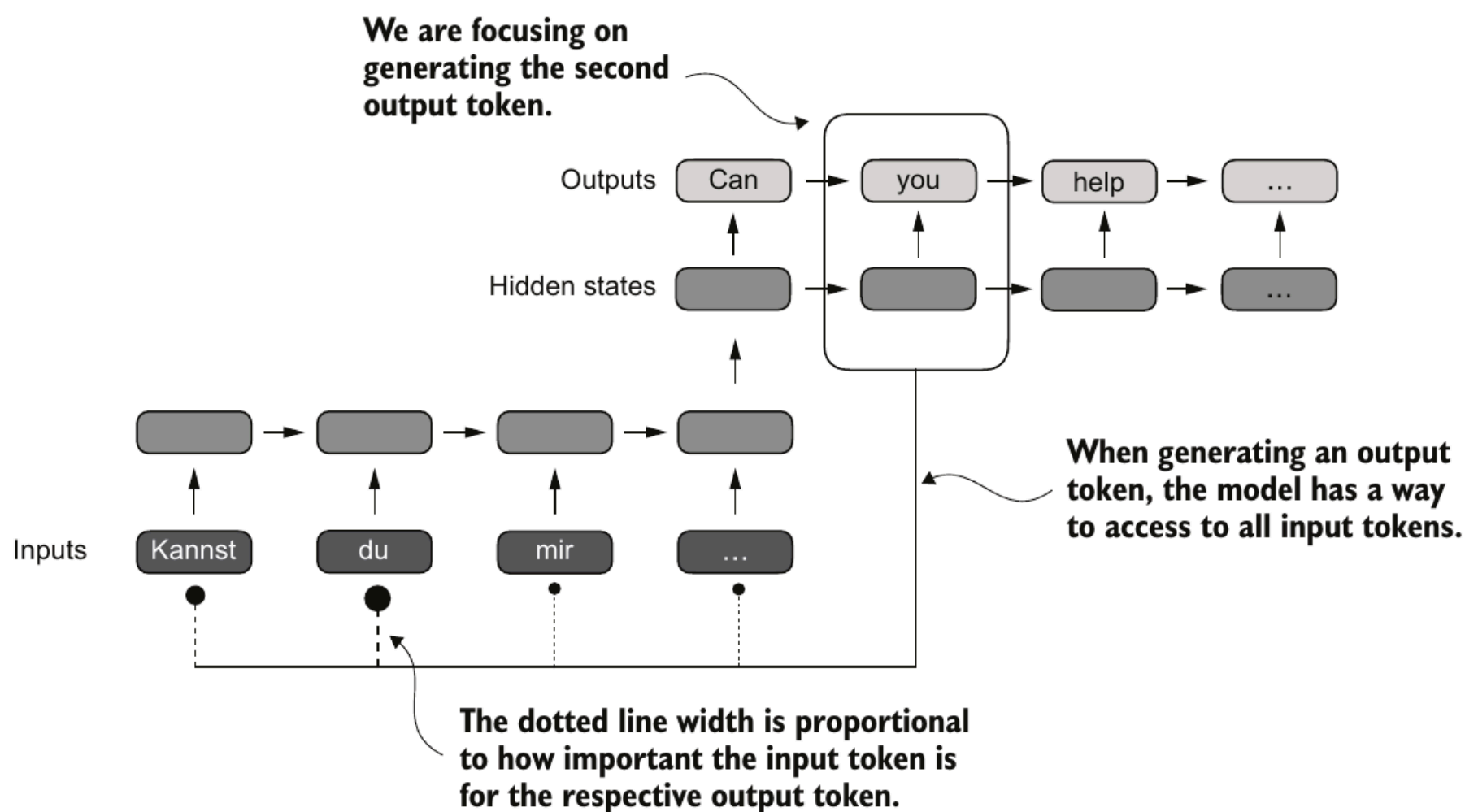
Attention의 선택적 입력 접근

해결 원리

- attention은 출력 하나를 만들 때 모든 입력 위치에 접근 가능한 경로를 둔다.
- 선의 굵기는 해당 출력에 대한 입력의 상대적 기여도를 나타낸다.
- 기여도는 수동 규칙이 아니라 뒤에서 계산할 attention weight로 정해진다.

관찰 기준

- 두 번째 출력 **you**가 여러 입력과 연결되고 연결 강도가 서로 다른지 확인한다.



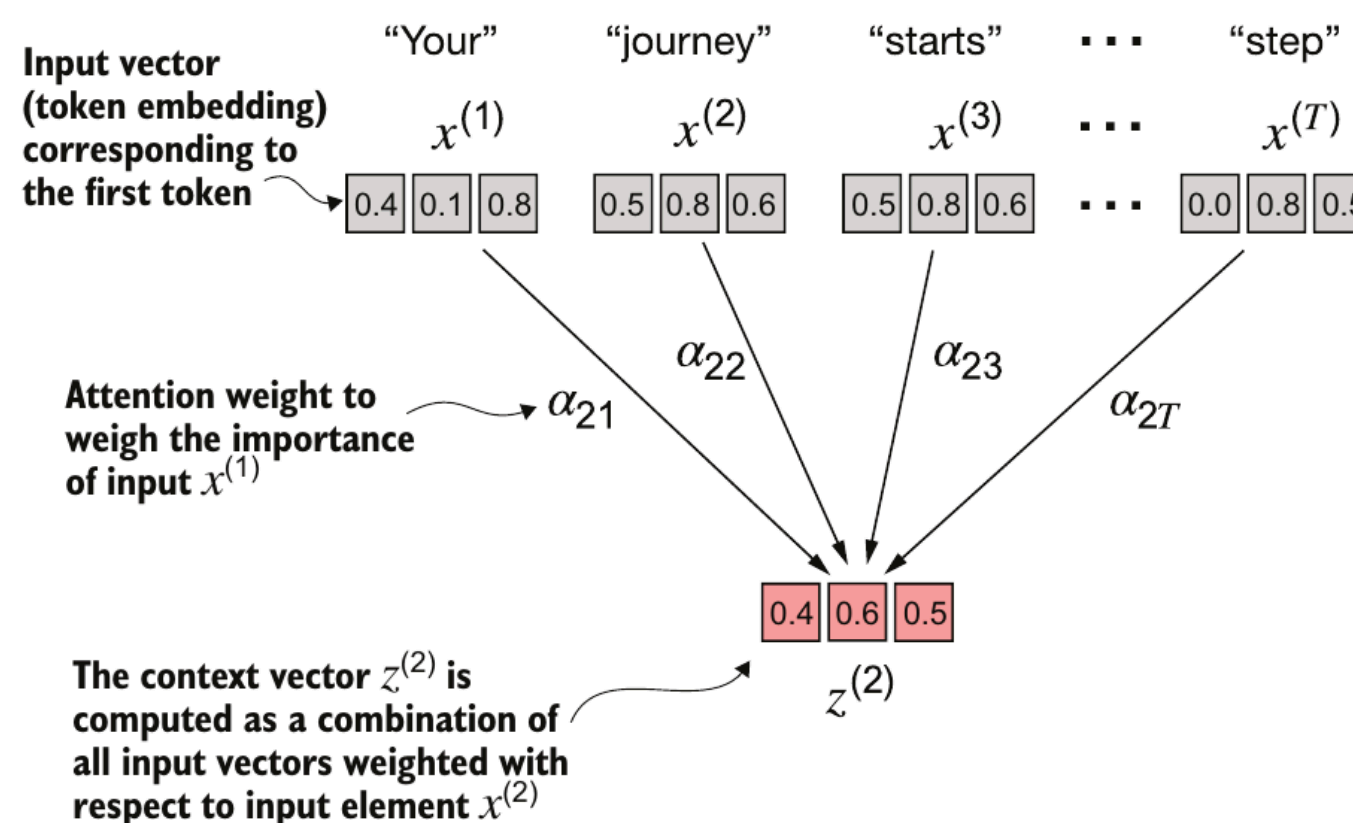
Self-Attention과 문맥 벡터

같은 문장 안의 비교

- query(질의)는 문맥 벡터를 만들 기준 위치의 벡터다. 두 번째 단어 journey의 벡터를 선택한다.
- self는 query와 비교·결합할 벡터가 같은 입력 X에서 나온다는 뜻이다. 번역 출력이 별도 원문을 읽는 경우와 구분한다.
- 그림의 위 입력 → 선의 기여 비율 → 아래 문맥 벡터 순서로 읽는다. 문맥 벡터는 입력들을 비율에 따라 섞은 결과다.

입력과 결과의 대응

- $x^{(j)}$ 는 j번째 입력, $z^{(2)}$ 는 두 번째 출력이다. α_{2j} 의 2는 기준 위치, j는 기여할 입력 위치다.
- 가운데 점은 생략된 입력을 뜻한다. 실제 계산은 여섯 입력 모두를 포함하며 결과는 후속 모델 연산에 전달한다.



05

고정 입력 임베딩 행렬

재현할 입력

- `attention_report.py`의 첫 부분이다. `import torch`는 PyTorch 계산 도구를 불러온다.
- `torch.tensor`는 6행 3열 실수 배열을 만든다. 행은 Your ... step 순서, 열은 특징 좌표다.
- 수치는 계산용으로 주어진 예시다. 실제 학습된 의미나 위치 임베딩을 계산한 결과라고 가정하지 않는다.

확인할 출력

- `shape`는 축 길이, `dtype`은 수의 저장 형식이다. `float32`는 32비트 실수다.

Python · `attention_report.py`

```
import torch

inputs = torch.tensor([
    [0.43, 0.15, 0.89], # Your
    [0.55, 0.87, 0.66], # journey
    [0.57, 0.85, 0.64], # starts
    [0.22, 0.58, 0.33], # with
    [0.77, 0.25, 0.10], # one
    [0.05, 0.80, 0.55], # step
])
print(inputs.shape, inputs.dtype)
```

실행 결과

```
torch.Size([6, 3]) torch.float32
```

Dot Product Attention Score

내적과 점수

- 내적(dot product)은 두 벡터의 같은 열끼리 곱한 뒤 합한 수다. 이 비교 점수를 attention score라 부른다.
- Your와 journey의 점수: $0.43 \times 0.55 + 0.15 \times 0.87 + 0.89 \times 0.66 = 0.9544$.
- 입력 [3] 두 개를 합산해 스칼라(수 하나)를 얻는다. 점수는 아직 기여 비율이 아니다.

해석의 한계

- 내적에는 방향과 벡터 길이가 함께 작용한다. query=[1,0]이면 [1,0]의 점수는 1, [2,2]는 2다.
- 점수가 더 큰 [2,2]가 query와 더 같은 방향은 아니다. 이 예시의 점수만으로 의미 유사도를 단정할 수 없다.

Attention Score의 실제 계산

한 토큰의 점수 계산

- `inputs[1]` 은 0부터 세어 두 번째 행을 query로 고른다. `inputs @ query` 는 여섯 내적을 계산한다.
- 그림의 위 입력과 초록 query를 비교해 파란 점수 ω_{2j} 하나씩을 얻는다. j 는 입력 위치다.
- 출력 [6]의 첫 값 0.9544가 손계산과 같다. 그림은 값을 짧게 잘라 표시하므로 아래 실제 수치로 계산한다.

Python · `attention_report.py`

```
query = inputs[1]
attn_scores_2 = inputs @ query
print(attn_scores_2)
```

실행 결과

```
tensor([0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865])
```

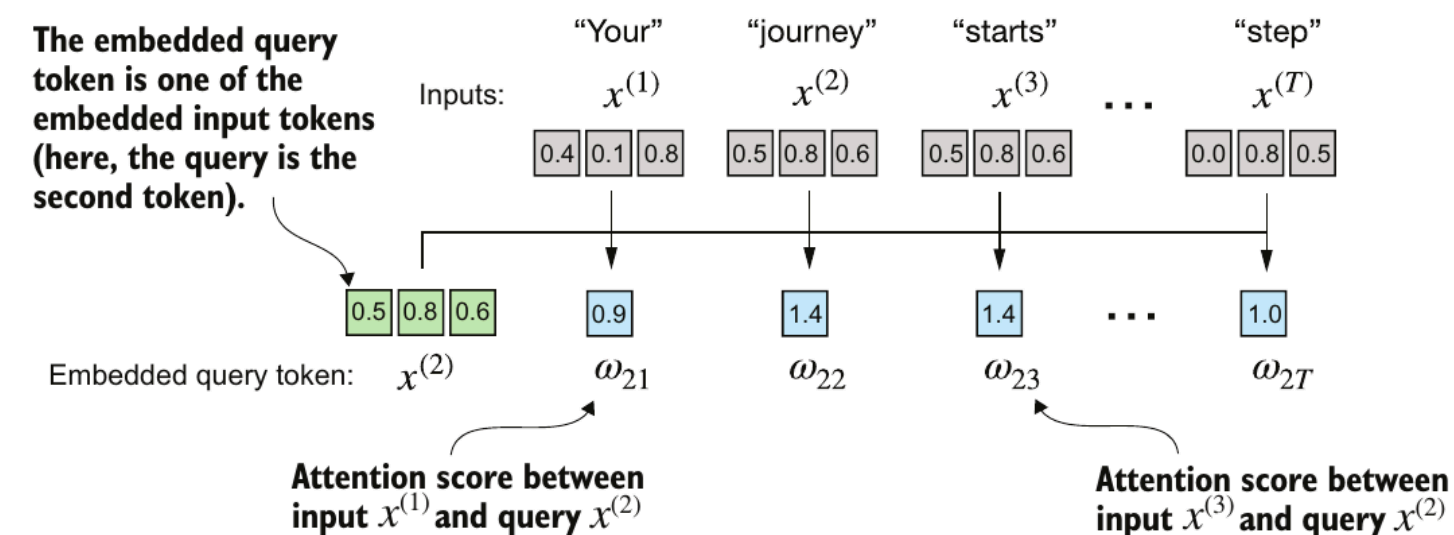


Figure 3.8 The overall goal is to illustrate the computation of the context vector $z^{(2)}$ using the second input element, $x^{(2)}$ as a query. This figure shows the first intermediate step, computing the attention scores ω between the query $x^{(2)}$ and all other input elements as a dot product. (Note that the numbers are truncated to one digit after the decimal point to reduce visual clutter.)

Score 정규화의 선택 기준

합으로 나눌 때의 실패

- $\text{score} = [-1, 2]$ 를 합 1로 나누면 $[-1, 2]$ 다. 합은 1이지만 음수는 기여 비율로 쓰기 어렵다.
- $\text{score} = [-1, 1]$ 이면 합이 0이라 나누기를 정의할 수 없다. 입력 점수에 부호가 섞일 수 있다.

Softmax의 해결과 경계

- softmax는 $a_j = e^{s_j} / \sum_k e^{s_k}$ 이다. s 는 점수, j, k 는 입력 위치, $e \approx 2.718$ 은 지수의 밑이다. $[-1, 2]$ 는 약 $[0.0474, 0.9526]$ 이 된다.
- 유한 점수의 정확한 수학값은 양수다. 컴퓨터에서는 매우 작은 비율이 0으로 저장될 수 있으므로 음수 없음과 행 합을 계산한다.
- 실제 코드는 안정적인 `torch.softmax`를 사용한다. 비율의 합 1은 의미적으로 올바른 연결의 증거가 아니다.

Softmax Weight의 수치 검증

점수에서 기여 비율로

- 그림의 파란 점수 ω_{2j} 를 정규화하면 노란 비율 α_{2j} 가 된다. `dim=0`은 이 1차원 벡터 전체를 계산 대상으로 삼는다.
- `sum()`은 모든 비율의 합이다. `allclose`는 두 값이 허용 오차 안에서 같은지 판정한다.
- `atol=1e-6`은 절대 오차 백만분의 1, `rtol=0`은 추가 상대 오차 없음이다. 계산에는 반올림 전 값을 사용한다.

Python · attention_report.py

```
attn_weights_2 = torch.softmax(attn_scores_2, dim=0)
print(attn_weights_2)
print(torch.allclose(attn_weights_2.sum(),
                      torch.tensor(1.), atol=1e-6, rtol=0))
```

실행 결과

```
tensor([0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581])
True
```

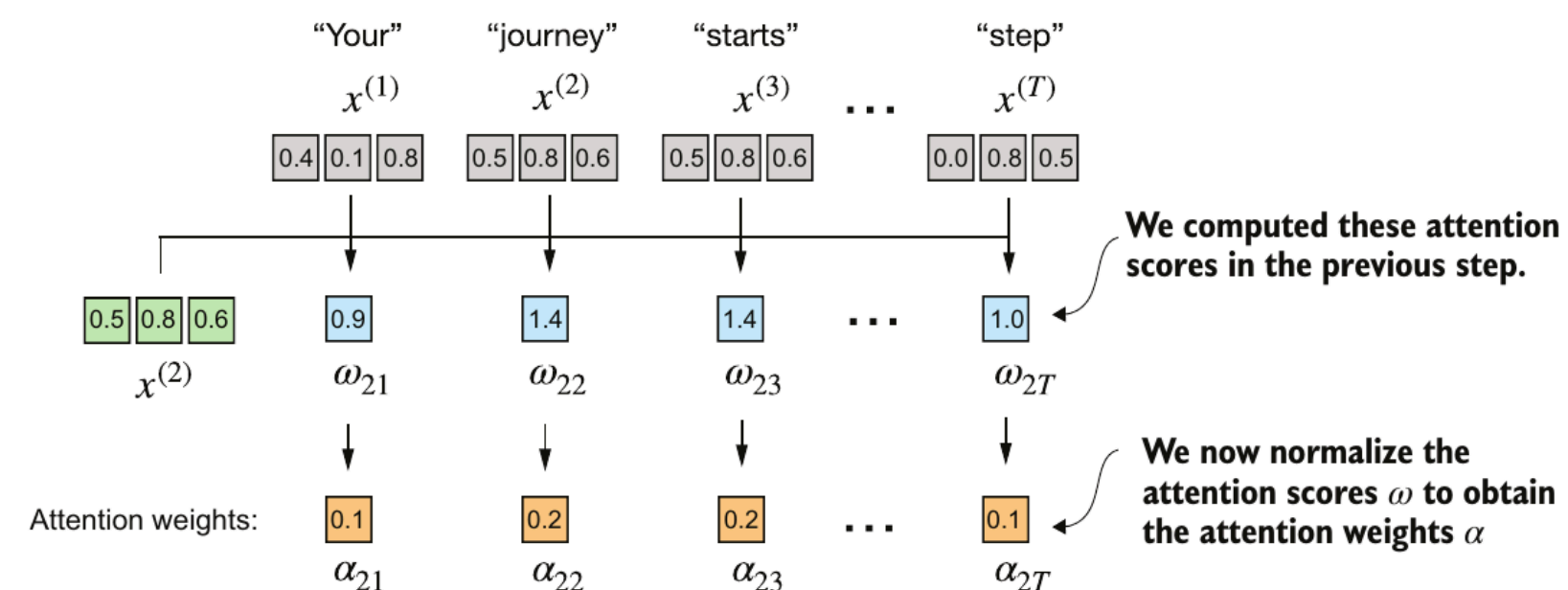


Figure 3.9 After computing the attention scores ω_{21} to ω_{2T} with respect to the input query $x^{(2)}$, the next step is to obtain the attention weights α_{21} to α_{2T} by normalizing the attention scores.

10

문맥 벡터의 가중합

같은 위치의 비율과 입력 결합

- 가중합은 각 비율 α_{2j} 를 같은 위치 입력의 세 좌표 모두에 곱한 뒤 좌표별로 더하는 연산이다.
- [6] @ [6,3]은 여섯 입력을 합쳐 세 특징을 남긴다. 비율의 위치와 입력 행의 짝을 유지해야 한다.
- 그림은 중간 입력을 생략한다. 마지막 비율의 α_{24} 표기는 α_{2T} 로 읽는다. 결과 [0.4,0.6,0.5]는 잘라 쓴 값이며 정상 반올림은 [0.4,0.7,0.6]이다.

Python · attention_report.py

```
context_vec_2 = attn_weights_2 @ inputs
print(context_vec_2)
```

실행 결과

```
tensor([0.4419, 0.6515, 0.5683])
```

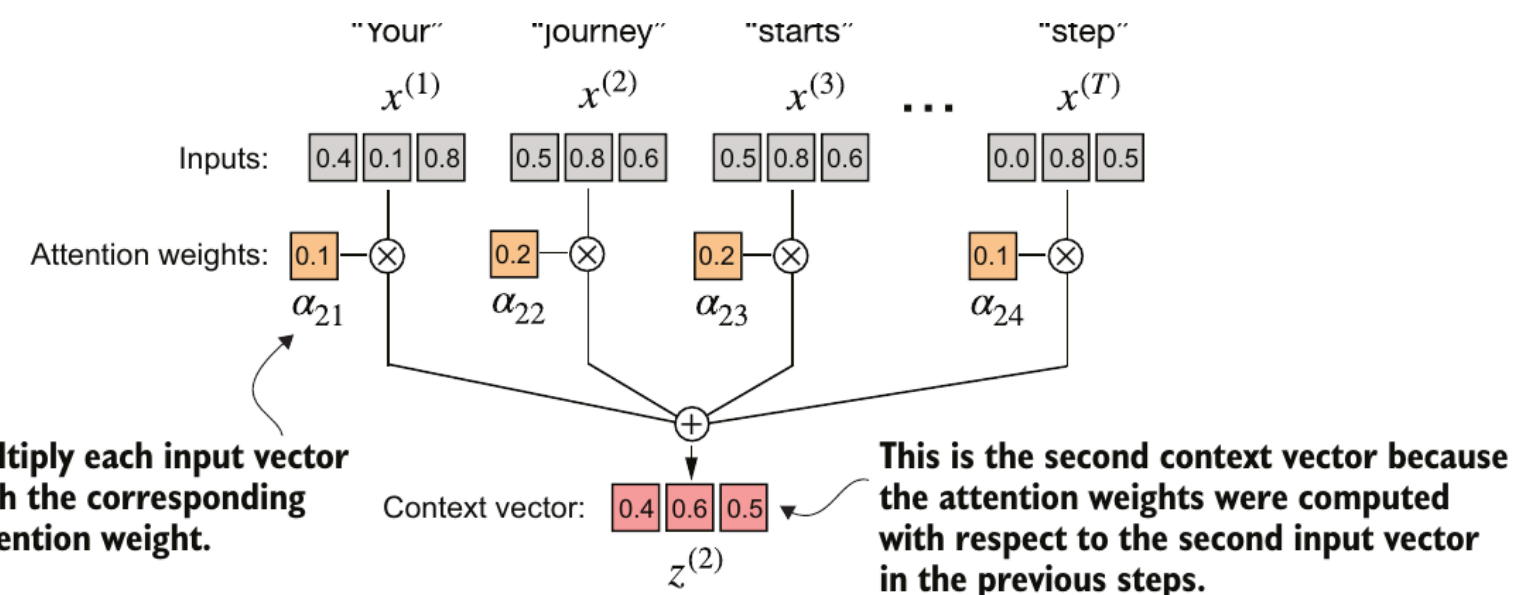


Figure 3.10 The final step, after calculating and normalizing the attention scores to obtain the attention weights for query $x^{(2)}$, is to compute the context vector $z^{(2)}$. This context vector is a combination of all input vectors $x^{(1)}$ to $x^{(T)}$ weighted by the attention weights.

11

개별 기여의 합산과 문맥 벡터

좌표별 합산

- `[:,None]` 은 여섯 비율 [6]을 [6,1]로 바꾼다. 콜론은 모든 행, None은 길이 1인 축 추가이다.
- `* inputs` 는 비율 하나를 같은 행의 세 좌표에 적용한다. 결과 [6,3]의 각 행은 입력 하나의 기여다.
- `sum(dim=0)` 은 기여 행 여섯 개를 합친다. `@` 와 같은 context 를 얻는다. 첫 좌표의 기여 약 [0.0596,0.1308,0.1330,0.0273,0.0833,0.0079]를 더하면 0.4419다.

Python · attention_report.py

```
contributions = attn_weights_2[:, None] * inputs
print(contributions.shape)
print(contributions.sum(dim=0))
print(torch.allclose(contributions.sum(dim=0),
                      context_vec_2, atol=1e-6, rtol=0))
```

실행 결과

```
torch.Size([6, 3])
tensor([0.4419, 0.6515, 0.5683])
True
```

Attention Weight 행렬의 축

행과 열의 의미

- 이제 한 query 계산을 여섯 query로 확장한다. 행 i는 기준 토큰, 열 j는 기여하는 입력 토큰이다.
- 노란 두 번째 행의 journey → starts 값은 약 0.2333이다. 반대 방향 starts → journey는 약 0.2369다.
- 그림은 소수 둘째 자리까지 잘라 표시한다. 표시값의 행 합이 1보다 작아도 실제 값으로 계산해야 한다.

	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18

This row contains the attention weights (normalized attention scores) computed previously

Figure 3.11 The highlighted row shows the attention weights for the second input element as a query. Now we will generalize the computation to obtain all other attention weights. (Please note that the numbers in this figure are truncated to two digits after the decimal point to reduce visual clutter. The values in each row should add up to 1.0 or 100%.)

13

모든 Score의 행렬 계산

한 query에서 전체 query로

- `inputs.T`는 $[6,3]$ 을 $[3,6]$ 으로 전치한다. `inputs @ inputs.T`는 토큰 쌍 36개의 내적을 $[6,6]$ 에 모은다.
- 셀 (i,j) 는 i 번째 기준 토큰과 j 번째 입력의 점수다. 같은 두 벡터의 내적이므로 점수 행렬은 대칭이다.
- 그림의 위 → 아래는 점수 계산, 행별 비율 계산, 문맥 벡터 계산이다. 지금은 첫 단계이며 아래 코드로 shape과 대칭을 검사한다.

Python · `attention_report.py`

```
attn_scores = inputs @ inputs.T
print(attn_scores.shape)
print(torch.allclose(attn_scores, attn_scores.T,
                      atol=1e-6, rtol=0))
```

실행 결과

```
torch.Size([6, 6])
True
```

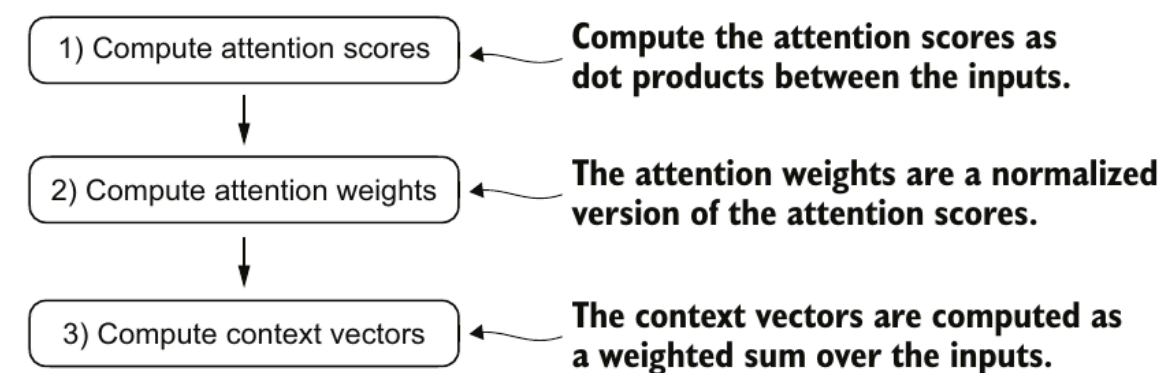


Figure 3.12 In step 1, we add an additional `for` loop to compute the dot products for all pairs of inputs.

14

행 단위 Softmax의 불변 조건

행마다 별도의 분포

- `dim=-1` 은 마지막 축, 즉 열을 따라 각 행을 따로 정규화한다. 기준 토큰 하나가 여섯 입력에 나눠 줄 비율이다.
- `sum(dim=-1)` 은 각 행을 더해 길이 6의 벡터를 만든다. `ones(6)` 은 1 여섯 개를 만들어 비교 기준으로 쓴다.
- 행 합이 1이어도 토큰 순서가 뒤섞일 수 있다. 뒤의 가중합에서는 열 j 와 입력 행 j 의 짝도 보존해야 한다.

Python · `attention_report.py`

```
attn_weights = torch.softmax(attn_scores, dim=-1)
print(attn_weights.sum(dim=-1))
print(torch.allclose(attn_weights.sum(dim=-1),
                     torch.ones(6), atol=1e-6, rtol=0))
```

실행 결과

```
tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
True
```

15

정규화 축 오류와 검증

같은 입력에서의 실패

- 각 query가 입력들에 나눠 줄 비율은 행 합이 1이어야 한다.
- `dim=0` 을 2차원 score에 쓰면 열마다 합이 1이 된다. shape은 맞아도 목표가 다르다.
- `dim=-1` 로 바꾸면 행마다 정규화된다. `False` 는 이 입력에서 행 합 조건이 실패했다는 뜻이다.

Python · `attention_report.py`

```
wrong = torch.softmax(attn_scores, dim=0)
print(torch.allclose(wrong.sum(dim=-1),
                    torch.ones(6), atol=1e-6, rtol=0))
print(torch.allclose(attn_weights.sum(dim=-1),
                    torch.ones(6), atol=1e-6, rtol=0))
```

실행 결과

```
False
True
```

16

점수 대칭과 기여 비율의 비대칭

서로 다른 정규화 기준

- score의 (i,j)와 (j,i)는 같은 내적이다. 하지만 softmax의 분모는 각 query 행 전체의 지수 합이다.
- journey → starts와 starts → journey의 점수는 1.4754로 같다. weight는 각 행의 다른 분모 때문에 달라진다.
- 이 비대칭은 오류가 아니다. 같은 score도 어떤 입력들과 함께 비교하는지에 따라 비율이 달라진다.

Python · attention_report.py

```
print(attn_scores[1, 2], attn_scores[2, 1])  
print(attn_weights[1, 2], attn_weights[2, 1])
```

실행 결과

```
tensor(1.4754) tensor(1.4754)  
tensor(0.2333) tensor(0.2369)
```

17

모든 문맥 벡터의 계산

행렬 곱과 결과

- `[6, 6] @ [6, 3]` 은 각 query 행의 weight로 여섯 입력을 가중합한다.
- 출력 두 번째 행은 단일 query로 계산한 `context_vec_2` 와 같아야 한다.
- `allclose` 가 `True` 이면 단일 계산을 전체 행렬로 일반화한 과정이 보존됐다.

Python · `attention_report.py`

```
all_context_vecs = attn_weights @ inputs
print(all_context_vecs.shape)
print(all_context_vecs[1])
print(torch.allclose(all_context_vecs[1], context_vec_2,
                      atol=1e-6, rtol=0))
```

실행 결과

```
torch.Size([6, 3])
tensor([0.4419, 0.6515, 0.5683])
True
```

18

단순 Self-Attention의 역할과 한계

확인한 결과

- $X: [6, 3] \rightarrow S: [6, 6] \rightarrow A: [6, 6] \rightarrow Z: [6, 3]$ 이며 두 번째 출력은 $[0.4419, 0.6515, 0.5683]$ 이다.
- A의 각 행은 현재 입력으로 계산한 기여 비율이다. 입력이 바뀌면 다시 계산한다.
- X를 고정한 이 연산에는 학습으로 갱신할 별도 파라미터가 없다.

이 결과로 말할 수 없는 것

- 행 합과 계산 일치는 구현을 검사하지만 실제 의미 이해나 번역 품질을 입증하지 않는다.
- 토큰별 출력이 여섯 행이라는 사실은 입력 정보를 손실 없이 복원할 수 있다는 뜻도 아니다.

Self-Attention 검증 리포트

동료에게 전달할 실행 파일

- `attention_report.py` 에 여섯 입력을 고정하고 score, weight, context를 순서대로 계산한다.
- 실행 결과에 토큰 순서와 A-Z의 shape, A 각 행의 합, 두 번째 context를 표시한다.
- `python attention_report.py` 는 현재 폴더의 소스를 읽어 실행하고 결과를 터미널에 표시한다.

최종 판정

- 두 번째 score 첫 값은 0.9544, context는 약 [0.4419,0.6515,0.5683]이어야 한다.
- 행 합과 단일·전체 계산의 일치는 반올림 전 값, 절대 오차 $1e-6$ 이하로 확인한다.

20

Summary

핵심 계산

- self-attention은 같은 입력에서 query를 고르고 다른 위치와 비교·결합한다.
- score는 내적 점수, weight는 행별로 정규화한 기여 비율, context는 입력 벡터의 가중합이다.
- 고정 입력에서 출력 여섯 행의 순서와 세 특징을 유지한다.

검산과 해석

- 각 행의 합과 음수 여부, 단일 query와 전체 행렬의 일치를 확인한다.
- 수치 검산 통과와 언어 의미 이해는 구분한다. 검증 리포트에는 실제 값과 허용 오차를 함께 남긴다.